



Computer Architecture

USER MANUAL

ARES - ARES Recursive Experimental System
Complete Guide for Building, Running, and Using the OS

Authors:

Juan Ignacio Raggio

Enzo Canelo

Matias Sanchez

Second Semester 2025

18/06/2026

Table of Contents

1	Introduction	4
1.1	About ARES	4
1.2	Document Purpose	4
2	Building and Compiling	5
2.1	Prerequisites	5
2.1.1	Required Software	5
2.1.2	Installing Docker	5
2.1.3	Installing QEMU	5
2.2	Compilation Scripts	5
2.2.1	compile_in_container.sh	5
2.2.2	clean_in_container.sh	6
2.3	Build Targets	7
2.3.1	all	7
2.3.2	kernel	7
2.3.3	userland	7
2.3.4	clean	7
2.4	Build Output	7
3	Running the Operating System	8
3.1	Using the Run Script	8
3.2	Debugging with GDB	8
4	Git Hooks Management	9
4.1	install-hooks.sh	9
4.2	setup-hooks.sh	9
5	Shell Commands Reference	10
5.1	help	10
5.2	man	10
5.3	inforeg	10
5.4	time	10
5.5	div	11
5.6	clear	11
5.7	printmem	11
5.8	history	12
5.9	exit	12
5.10	cursor	12
5.11	tron	12
5.12	textcolor	13
5.13	bgcolor	13
5.14	benchmark	14
6	Troubleshooting	15
6.1	Compilation Issues	15
6.1.1	Docker container not found	15
6.1.2	Permission denied errors	15
6.2	Runtime Issues	15
6.2.1	QEMU not found	15
6.2.2	Image file not found	15
6.2.3	Black screen on boot	16

6.3	Development Issues	16
6.3.1	clang-format not found	16
6.3.2	Validation script fails	16
7	Quick Reference	17
7.1	Common Command Sequences	17
7.2	System Specifications	17

1 Introduction

1.1 About ARES

ARES (ARES Recursive Experimental System) is an educational operating system kernel developed from scratch for the x86-64 architecture. This project demonstrates fundamental concepts of computer architecture, low-level programming, and operating system design.

The system features:

- Custom bootloader based on Pure64
- Kernel running in 64-bit long mode
- Hardware drivers (video, keyboard, timer, sound)
- Interactive shell with built-in commands
- Tron game as a demonstration application

1.2 Document Purpose

This manual provides comprehensive instructions for:

1. Building and compiling the ARES operating system
2. Running the OS in QEMU emulator
3. Managing the development environment
4. Using the shell and its commands
5. Playing the Tron game

2 Building and Compiling

2.1 Prerequisites

Before building ARES, ensure you have the following tools installed:

2.1.1 Required Software

- **Docker:** Container runtime for the build environment
- **Bash:** Shell scripting (usually pre-installed on Unix-like systems)
- **QEMU:** x86-64 system emulator for running the OS
- **Git:** Version control (for managing the repository)

Important: All compilation must be done inside a Docker container to ensure a consistent build environment with the correct toolchain (GCC cross-compiler for x86-64 bare metal).

2.1.2 Installing Docker

```
# macOS
brew install docker

# Ubuntu/Debian
sudo apt-get install docker.io

# Arch Linux
sudo pacman -S docker
```

2.1.3 Installing QEMU

```
# macOS
brew install qemu

# Ubuntu/Debian
sudo apt-get install qemu-system-x86

# Arch Linux
sudo pacman -S qemu
```

2.2 Compilation Scripts

2.2.1 compile_in_container.sh

This script compiles the project inside a Docker container.

Usage:

```
./AresOS/compile_in_container.sh [CONTAINER_NAME] [TARGET]
```

Parameters:

- `CONTAINER_NAME` : Name of the Docker container (default: `ARES`)
- `TARGET` : Make target to build (default: `all`)

Examples:

```
# Compile everything using default container
./AresOS/compile_in_container.sh

# Compile using a specific container
./AresOS/compile_in_container.sh my-ares-container

# Compile only the kernel
./AresOS/compile_in_container.sh ARES kernel

# Compile only userland
./AresOS/compile_in_container.sh ARES userland
```

What it does:

1. Verifies Docker is installed and running
2. Checks if the specified container exists
3. Starts the container if it's not running
4. Executes `make [TARGET]` inside the container
5. Fixes file permissions after compilation
6. Reports success or failure

Note: The script automatically corrects file permissions of generated files, as Docker containers often create files with root ownership.

2.2.2 clean_in_container.sh

This script cleans all build artifacts from the project.

Usage:

```
./AresOS/clean_in_container.sh [CONTAINER_NAME]
```

Parameters:

- `CONTAINER_NAME` : Name of the Docker container (default: `ARES`)

Examples:

```
# Clean using default container
./AresOS/clean_in_container.sh

# Clean using specific container
./AresOS/clean_in_container.sh my-ares-container
```

What it does:

This script is a wrapper around `compile_in_container.sh` that executes the `clean` make target. It removes:

- Object files (`.o`)
- Binary files
- Build directories
- Generated images
- Temporary files

Tip: Always run `clean_in_container.sh` before making significant changes to the build system or when switching between different development branches.

2.3 Build Targets

The Makefile supports several targets:

2.3.1 all

Builds the complete system: kernel, userland, and bootable image.

```
./AresOS/compile_in_container.sh  
# or  
./AresOS/compile_in_container.sh ARES all
```

2.3.2 kernel

Builds only the kernel binary.

```
./AresOS/compile_in_container.sh ARES kernel
```

2.3.3 userland

Builds only the userland applications (shell and programs).

```
./AresOS/compile_in_container.sh ARES userland
```

2.3.4 clean

Removes all build artifacts.

```
./AresOS/clean_in_container.sh
```

2.4 Build Output

- `AresOS/Image/x64BareBonesImage.img` : Raw bootable disk image
- `AresOS/Kernel/bin/` : Kernel binaries
- `AresOS/Kernel/build/` : Kernel object files
- `AresOS/Userland/*/bin/` : Userland binaries

3 Running the Operating System

3.1 Using the Run Script

The simplest way to run ARES is using the provided run script.

Usage:

```
./AresOS/run.sh [-d]
```

Options:

- **-d** : Run in debug mode (waits for GDB connection)

Examples:

```
# Run normally
./AresOS/run.sh

# Run in debug mode
./AresOS/run.sh -d
```

Note: The script automatically selects the QCOW2 image if available, otherwise it uses the raw IMG file.

Note: You can also run QEMU manually with custom options

3.2 Debugging with GDB

To debug the kernel with GDB:

1. Start QEMU in debug mode:

```
./AresOS/run.sh -d
```

2. In another terminal, connect GDB:

```
gdb AresOS/Kernel/bin/kernel.elf
(gdb) target remote localhost:1234
(gdb) continue
```

Tip: Add breakpoints before running `continue` :

```
(gdb) break kernel_main
(gdb) continue
```


4 Git Hooks Management

Git hooks help maintain code quality by automatically running checks before commits.

4.1 install-hooks.sh

This script installs Git pre-commit hooks that automatically format code.

Usage:

```
./install-hooks.sh
```

What it does:

1. Verifies you're in a Git repository
2. Checks if `clang-format` is installed
3. Creates a pre-commit hook that:
 - Detects staged `.c` and `.h` files
 - Automatically formats them using `clang-format`
 - Re-stages the formatted files
 - Commits the formatted code

Installing clang-format:

```
# macOS
brew install clang-format

# Ubuntu/Debian
sudo apt-get install clang-format

# Arch Linux
sudo pacman -S clang
```

Note: Each developer must run this script on their local machine. Git hooks are not automatically shared via the repository.

4.2 setup-hooks.sh

This script configures Git to use a custom hooks directory.

Usage:

```
./setup-hooks.sh
```

What it does:

1. Verifies the repository structure
2. Configures Git to use the `.githubhooks` directory
3. Lists available hooks
4. Displays confirmation message

5 Shell Commands Reference

Once ARES boots, you will be presented with an interactive shell prompt:

```
>
```

This section describes all available commands.

5.1 help

Displays a list of all available commands with brief descriptions.

Syntax:

```
help
```

Parameters: None

Note: The commands `history` and `cursor` are not yet implemented in the current version.

5.2 man

Shows detailed manual information for a specific command.

Syntax:

```
man <command>
```

Parameters:

- `command` : Name of the command to get information about

5.3 infoREG

Displays a snapshot of all CPU registers at the moment of the last register capture.

Syntax:

```
infoREG
```

Parameters: None

Note: Register values are captured through a syscall that takes a snapshot of the current register state.

5.4 time

Shows the current system time and elapsed time since shell startup.

Syntax:

```
time
```

Parameters: None

Note: Time is read from the Real-Time Clock (RTC), it has a 3 hour gap with Argentina's current time. The elapsed time is calculated from when the shell was first started.

5.5 div

Performs integer division of two numbers.

Syntax:

```
div <numerator> <denominator>
```

Parameters:

- **numerator**: The number to be divided (dividend)
- **denominator**: The number to divide by (divisor)

Warning: Division by zero is not handled gracefully so it will cause a system exception.

5.6 clear

Clears the entire screen and moves the cursor to the top-left position.

Syntax:

```
clear
```

Parameters: None

5.7 printmem

Displays a hexadecimal dump of 32 bytes from a specified memory address.

Syntax:

```
printmem <address>
```

Parameters:

- **address**: Memory address in hexadecimal format (with or without **0x** prefix)

Warning: Reading from invalid or protected memory addresses may cause the system to have unexpected problems

5.8 history

Warning: This command is not yet implemented in the current version of ARES.

Displays the command history showing previously executed commands.

Syntax:

```
history
```

Parameters: None

5.9 exit

Warning: We need Scheduler to make this work

Exits the ARES operating system

Syntax:

```
exit
```

Parameters: None

5.10 cursor

Warning: This command is not yet implemented in the current version of ARES.

Changes the cursor shape in the shell.

Syntax:

```
cursor <shape>
```

Parameters:

- **shape**: One of: `block`, `hollow`, `line`, or `underline`

5.11 tron

Launches the Tron light cycle game.

Syntax:

```
tron
```

Parameters: None

Game Controls:

Player 1 (Cyan):

- **W** : Move up
- **A** : Move left
- **S** : Move down
- **D** : Move right

Player 2 (Magenta):

- **I** : Move up
- **J** : Move left
- **K** : Move down
- **L** : Move right

To Exit:

Press **ESC** to return to the shell.

5.12 textcolor

Changes the foreground text color of the shell.

Syntax:

```
textcolor <color>
```

Parameters:

- **color** : Color name or hexadecimal RGB value

Supported color names:

- black , white , red , green , blue
- yellow , cyan , magenta
- gray , lightgray , darkgray

Note: The text color affects all subsequent text output in STDOUT. The color persists until changed again or the system is restarted.

5.13 bgcolor

Changes the background color of the shell.

Syntax:

```
bgcolor <color>
```

Parameters:

- **color** : Color name or hexadecimal RGB value

Supported color names:

- black , white , red , green , blue
- yellow , cyan , magenta
- gray , lightgray , darkgray

Tip: Combine `textcolor` and `bcolor` to create custom color schemes. Use the `clear` command after changing colors to see the full effect.

5.14 benchmark

Runs a comprehensive performance benchmark suite testing various system components.

Syntax:

```
benchmark
```

Parameters: None

What it tests:

1. **FPS (Frames Per Second) Benchmark**
2. **Timer Benchmark**
3. **Syscall Read Benchmark**

Note: The benchmark requires user interaction to advance between tests. Press any key when prompted to continue to the next benchmark.

6 Troubleshooting

6.1 Compilation Issues

6.1.1 Docker container not found

Error:

```
ERROR: The container 'ARES' does not exist
```

Solution: Create a Docker container or use the correct container name.

6.1.2 Permission denied errors

Error:

```
Permission denied when accessing files in Kernel/build/
```

Solution: The compile script should fix this automatically, but you can manually fix permissions:

```
sudo chown -R $(id -u):$(id -g) AresOS/Kernel/build
sudo chown -R $(id -u):$(id -g) AresOS/Image
```

6.2 Runtime Issues

6.2.1 QEMU not found

Error:

```
qemu-system-x86_64: command not found
```

Solution: Install QEMU using your package manager (see Prerequisites section).

6.2.2 Image file not found

Error:

```
Could not open 'Image/x64BareBonesImage.img'
```

Solution: Compile the project first:

```
./AresOS/compile_in_container.sh
```

6.2.3 Black screen on boot

Possible causes:

1. QEMU not properly configured for graphics
2. Bootloader not properly installed
3. Kernel binary corrupted

Solution: Try rebuilding:

```
./AresOS/clean_in_container.sh
./AresOS/compile_in_container.sh
./AresOS/run.sh
```

6.3 Development Issues

6.3.1 clang-format not found

Error when installing hooks:

```
Warning: clang-format is not installed
```

Solution: Install clang-format (see Git Hooks section).

6.3.2 Validation script fails

Error:

```
VALIDATION FAILED: X errors found
```

Solution: Review the error messages and fix the reported issues. Common problems:

- Duplicate header files
- Relative path includes
- Missing directories
- Missing `compile_flags.txt` files

7 Quick Reference

7.1 Common Command Sequences

Build and run:

```
./AresOS/compile_in_container.sh  
./AresOS/run.sh
```

Clean and rebuild:

```
./AresOS/clean_in_container.sh  
./AresOS/compile_in_container.sh
```

Validate and build:

```
./AresOS/validate_structure.sh  
./AresOS/compile_in_container.sh
```

Setup development environment:

```
./install-hooks.sh  
./setup-hooks.sh
```

7.2 System Specifications

Architecture: x86-64 (64-bit)

Bootloader: Pure64

Memory: 512 MB RAM (configurable)

Drivers:

- Video (framebuffer)
- Keyboard
- Timer/RTC
- PC Speaker (sound)